Cycle 11/03

Anna Hale

2026-11-02

Function 1

```
hand_value <- function(hand) { # function with hand as argument
  ranks <- sort(as.numeric(hand$rank)) # created column vector of all of the ranks sorted by rank
  distinct_ranks <- length(unique(hand$rank)) # counted the number of distinct ranks
  straight <- max(ranks) - min(ranks) == 4 || identical(ranks, c(2, 3, 4, 5, 14)) # T/F value if hand i
  flush <- all(hand$suit == hand$suit[1]) # T/F value if hand is flush (suits are identical)
  if (flush == TRUE) {
    if (straight == TRUE) {
      if (identical(ranks, c(10, 11, 12, 13, 14))) {
        return("Royal Flush")
        # if hand is flush, straight, and 10-A, return royal flush
      }
      else {
        return("Straight Flush")
        # if hand is flush, straight, and not a royal flush, return straight flush
      }
    }
    else {
      return("Flush")
      # if hand is a flush but not a straight, return flush
    }
  }
  else if (distinct_ranks == 2) { # if there are only 2 types of ranks in the hand
    if (ranks[1] == ranks[4] || ranks[2] == ranks[5]) {
      return("Four of a Kind")
      # if 4 of the cards are the same rank, return 4 of a kind
    }
    else {
      return("Full House")
      # if two distinct ranks in the hand and not 4 of a kind, return full house (must be 3 of same ran
    }
  }
  else if (straight == TRUE) {
    return("Straight")
    # if hand does not meet previous conditions and is a straight, return straight
  }
  else if (distinct_ranks == 3) { # if hand has three different types of rank
    count <- table(ranks) # count the number of cards is each rank
    if (max(count) == 3) {
```

```
      return("Three of a Kind")
      # if hand has 3 of the same rank, return three of a kind
    }
    else {
      return("Two Pair")
      # if not three of a kind, must be 2 pair
    }
  }
  else if (distinct_ranks == 4) {
    return("One Pair")
    # if there are 4 distinct ranks, two must be same rank, therefore return one pair
  }
  else {
    return("High Card")
    # if none of these conditions are met, return high card
  }
}
```

test cases

```
royal_flush <- data.frame( # royal flush test case
  rank = c("10", "11", "12", "13", "14"),
  suit = c("H", "H", "H", "H", "H")
)
straight_flush <- data.frame( # straight flush test case
  rank = c("5", "6", "7", "8", "9"),
  suit = c("S", "S", "S", "S", "S")
)
four_of_a_kind <- data.frame( # four of a kind test case
  rank = c("8", "8", "8", "8", "2"),
  suit = c("C", "D", "H", "S", "H")
)
full_house <- data.frame( # full house test case
  rank = c("12", "12", "12", "4", "4"),
  suit = c("C", "D", "H", "S", "C")
)
flush <- data.frame( # flush test case
  rank = c("2", "5", "9", "11", "14"),
  suit = c("D", "D", "D", "D", "D")
)
straight <- data.frame( # straight test case
  rank = c("4", "5", "6", "7", "8"),
  suit = c("H", "C", "S", "D", "H")
)
straight_ace_low <- data.frame( # straight where ace is low
  rank = c("14", "2", "3", "4", "5"),
  suit = c("H", "C", "S", "D", "H")
)
three_kind <- data.frame( # three of a kind test case
  rank = c("7", "7", "7", "2", "4"),
  suit = c("H", "C", "S", "D", "H")
)
```

```r
two_pair <- data.frame( # two pair test case
  rank = c("11", "11", "5", "5", "14"),
  suit = c("H", "C", "S", "D", "H")
)
one_pair <- data.frame( # one pair test case
  rank = c("10", "10", "3", "5", "9"),
  suit = c("H", "C", "S", "D", "H")
)
high_card <- data.frame( # high card test case
  rank = c("14", "11", "8", "4", "2"),
  suit = c("H", "C", "S", "D", "C")
)
test_cases <- list(
  Royal_Flush     = royal_flush,
  Straight_Flush  = straight_flush,
  Four_Kind       = four_of_a_kind,
  Full_House      = full_house,
  Flush           = flush,
  Straight    = straight,
  Straight_Ace  = straight_ace_low,
  Three_Kind      = three_kind,
  Two_Pair        = two_pair,
  One_Pair        = one_pair,
  High_Card       = high_card
)

test_results <- lapply(test_cases, hand_value)
print(test_results)
```

```
## $Royal_Flush
## [1] "Royal Flush"
##
## $Straight_Flush
## [1] "Straight Flush"
##
## $Four_Kind
## [1] "Four of a Kind"
##
## $Full_House
## [1] "Full House"
##
## $Flush
## [1] "Flush"
##
## $Straight
## [1] "Straight"
##
## $Straight_Ace
## [1] "Straight"
##
## $Three_Kind
## [1] "Three of a Kind"
##
```

```
## $Two_Pair
## [1] "Two Pair"
##
## $One_Pair
## [1] "One Pair"
##
## $High_Card
## [1] "High Card"
```

Function 2

```
hand_score <- function(hand) { # created function with hand as argument
  if (nrow(hand) == 5) { # if 5-card hand, return the hand with the value of hand
    return(list(hand_value(hand), hand))
  }
  else {
    hand_order <- c("High Card", "One Pair", "Two Pair", "Three of a Kind",
                    "Straight", "Flush", "Full House", "Four of a Kind",
                    "Straight Flush", "Royal Flush") # created way to order the hands
    combs <- combn(nrow(hand), 5) # all possible combinations
    best_rank <- 0
    best_value <- NULL
    best_subset <- NULL # initalized variables for the best rank, value, subset
    for (i in 1:ncol(combs)) { # going through all cominations
      current_subset <- hand[combs[, i], ] # found the current subset we are evaluating
      value <- as.character(hand_value(current_subset)) # found the value of the current subset
      rank <- match(value, hand_order) # found the "rank" of the value of the subset
      if (rank > best_rank) { # if this rank if better than previous, it replaces the best rank
        best_rank <- rank
        best_value <- value
        best_subset <- current_subset
      }
    }
    return(list(best_value, best_subset)) # return best rank
  }
}
```

test cases

```
royal_7 <- data.frame(
  rank = c(10, 11, 12, 13, 14, 2, 3),
  suit = c("H", "H", "H", "H", "H", "C", "D")
)
fullhouse_6 <- data.frame(
  rank = c(12, 12, 12, 4, 4, 7),  # Three 12s (Queens), Two 4s, one extra 7
  suit = c("C", "D", "H", "S", "C", "S")
)
flush_7 <- data.frame(
  rank = c(2, 5, 9, 11, 14, 7, 7),  # 2, 5, 9, 11, 14 of Diamonds (a Flush) + two 7s
  suit = c("D", "D", "D", "D", "D", "H", "C")
```

```
)
test_cases <- list(
  royal_7,
  fullhouse_6,
  flush_7
)
test_results <- lapply(test_cases, hand_score) # applying function to all of test cases
print(test_results)
```

```
## [[1]]
## [[1]][[1]]
## [1] "Royal Flush"
##
## [[1]][[2]]
##   rank suit
## 1   10    H
## 2   11    H
## 3   12    H
## 4   13    H
## 5   14    H
##
##
## [[2]]
## [[2]][[1]]
## [1] "Full House"
##
## [[2]][[2]]
##   rank suit
## 1   12    C
## 2   12    D
## 3   12    H
## 4    4    S
## 5    4    C
##
##
## [[3]]
## [[3]][[1]]
## [1] "Flush"
##
## [[3]][[2]]
##   rank suit
## 1    2    D
## 2    5    D
## 3    9    D
## 4   11    D
## 5   14    D
```